

Tutorial: Despliegue de una Aplicación Web Básica en Elixir



elixir

Asignatura:

UT2-A1 – Despliegue de Aplicaciones Web

Realizado por:

→ *Braulio Herrera, Oliver Rodriguez, Javier Paredes.*

Curso:

2º ASIRM

Fecha:

20 de octubre de 2025

Introducción

Este tutorial guía paso a paso cómo crear, desplegar y mantener una aplicación web básica en **Elixir**, usando **Plug** y **Cowboy** en **Ubuntu Server 24.04**.

La aplicación mostrará información dinámica del cliente (IP, navegador, resolución de pantalla, hora, versión de Elixir, etc.)

Además, se documentan los errores encontrados durante el proceso y cómo solucionarlos.

1 Introducción al lenguaje Elixir

Elixir es un lenguaje **funcional, concurrente y compilado** sobre la **máquina virtual de Erlang (BEAM)**, conocido por su tolerancia a fallos y su capacidad para manejar miles de conexiones simultáneas.

- Compila a **bytecode de Erlang**.
 - Ideal para **aplicaciones web en tiempo real**.
 - Suele usarse con **Plug + Cowboy** o el framework **Phoenix** para aplicaciones más completas.
-

2 Prerrequisitos

Antes de comenzar asegúrate de tener:

- Ubuntu Server 24.04 con acceso sudo.
 - Acceso SSH remoto (por ejemplo desde VSCode).
 - VSCode o VSCodium con la extensión **Remote - SSH**.
 - Conexión a Internet para instalar dependencias.
-

3 Instalación de Elixir y dependencias

```
sudo apt update
sudo apt install elixir -y
```

Verifica la instalación con:

```
elixir -v
```

4 Crear el proyecto base

```
mix new webapp --sup
cd webapp
```

El parámetro `--sup` genera una aplicación **supervisada**, ideal para ejecutarse como servicio.

5 Añadir dependencias Plug y Cowboy

Edita el archivo `mix.exs` y reemplaza el bloque `deps` por:

```
defp deps do
  [
    {:plug_cowboy, "~> 2.6"}
  ]
end
```

Luego instala las dependencias:

```
mix deps.get
```

6 Configurar el servidor en **Application**

Edita el archivo `lib/webapp/application.ex` y déjalo así:

```
defmodule Webapp.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
```

```

        {Plug.Cowboy, scheme: :http, plug: Webapp.Router, options:
[port: 8080]}
    ]

    opts = [strategy: :one_for_one, name: Webapp.Supervisor]
    Supervisor.start_link(children, opts)
end
end

```

7 Crear el router principal

Crea el archivo `lib/webapp/router.ex` y pega el siguiente código mejorado (versión visual y funcional):

```

defmodule Webapp.Router do
  use Plug.Router

  plug :match
  plug :dispatch

  get "/" do
    {version, _} = System.cmd("elixir", ["-v"])
    user_agent = get_req_header(conn, "user-agent") |> List.first()
    ip = Tuple.to_list(conn.remote_ip) |> Enum.join(".")
    now = DateTime.utc_now() |> DateTime.to_string()

    html = """
    <!DOCTYPE html>
    <html lang="es">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width,
initial-scale=1.0">
      <title>🌐 Información del Cliente</title>
      <style>
        body {
          font-family: 'Segoe UI', Arial, sans-serif;
          background: #eef3f9;
          margin: 0;
          padding: 0;

```

```

        color: #222;
    }
    header {
        background: #007acc;
        color: white;
        padding: 20px;
        text-align: center;
        box-shadow: 0 2px 5px rgba(0,0,0,0.2);
    }
    main { max-width: 900px; margin: 40px auto; padding: 0 20px;
}

    section {
        background: white;
        margin: 20px 0;
        border-radius: 12px;
        padding: 25px;
        box-shadow: 0 4px 10px rgba(0,0,0,0.05);
        border-left: 6px solid #007acc;
    }
    h2 { color: #004b87; border-bottom: 1px solid #cce0f5;
padding-bottom: 6px; }
    footer {
        text-align: center; padding: 20px; color: #666;
        border-top: 1px solid #ccc; font-size: 0.9rem;
    }
</style>
</head>
<body>
    <header><h1> Información del Cliente</h1></header>
    <main>
        <section>
            <h2>Datos del Servidor</h2>
            <p><b>Fecha y hora:</b> #{now}</p>
            <p><b>Versión de Elixir:</b></p>
            <pre>#{version}</pre>
            <p><b>Hora local:</b> <span id="clock"></span></p>
        </section>
        <section>
            <h2>Datos del Cliente</h2>
            <p><b>IP:</b> #{ip}</p>
            <p><b>Navegador:</b> #{user_agent}</p>

```

```

    <p><b>Resolución:</b> <span id="resol"></span></p>
    <p><b>Sistema operativo:</b> <span id="so"></span></p>
    <p><b>Tipo de dispositivo:</b> <span
id="device"></span></p>
  </section>
</main>
<footer>
  Aplicación desarrollada en Elixir 1.14 con Plug + Cowboy ©
2025
</footer>
<script>
  // Reloj dinámico
  function actualizarReloj() {
    const ahora = new Date();
    document.getElementById("clock").innerText =
      ahora.toLocaleTimeString("es-ES", { hour12: false });
  }
  setInterval(actualizarReloj, 1000);
  actualizarReloj();

  // Resolución
  document.getElementById("resol").innerText =
    window.screen.width + " x " + window.screen.height;

  // Sistema operativo
  const ua = navigator.userAgent;
  let so = "Desconocido";
  if (ua.includes("Win")) so = "Windows";
  else if (ua.includes("Mac")) so = "macOS";
  else if (ua.includes("Linux")) so = "Linux";
  else if (ua.includes("Android")) so = "Android";
  else if (ua.includes("like Mac")) so = "iOS";
  document.getElementById("so").innerText = so;

  // Tipo de dispositivo
  const isMobile = /Mobi|Android/i.test(ua);
  document.getElementById("device").innerText = isMobile ?
"Móvil / Tablet" : "PC / Escritorio";
</script>
</body>
</html>

```

```
    ""

    send_resp(conn, 200, html)
  end

  match _ do
    send_resp(conn, 404, "Ruta no encontrada")
  end
end
```

8 Ejecutar la aplicación

```
mix run --no-halt
```

Luego abre en el navegador:

```
http://IP_DEL_SERVIDOR:8080
```

9 Configurar como servicio **systemd**

Crea el servicio:

```
sudo nano /etc/systemd/system/webapp.service
```

Contenido:

```
[Unit]
Description=Elixir WebApp
After=network.target

[Service]
User=nobody
WorkingDirectory=/home/user/webapp
ExecStart=/usr/bin/mix run --no-halt
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Activar y arrancar el servicio:

```
sudo systemctl daemon-reload
sudo systemctl start webapp
sudo systemctl enable webapp
```

10 Configurar SSH (para edición remota)

```
sudo apt install openssh-server -y
sudo systemctl enable ssh
sudo systemctl start ssh
```

Conéctate desde VSCode usando la extensión **Remote - SSH** con la IP del servidor.

11 Problemas encontrados y soluciones

✗ Error: “Could not find rebar3...”

Causa: Mix no puede descargar metadatos de Hex.

Solución:

```
mix local.hex --force
mix local.rebar --force
mix deps.get
```

✗ Error HTTP 500

Causa: Error en el uso de `System.cmd("elixir", ["-v"])`.

Solución: Cambiar `{:ok, version}` por `{version, _}` para capturar correctamente la salida.

✗ Acentos o emojis rotos

Solución: Añadir `<meta charset="UTF-8">` en la cabecera HTML.

✗ No conexión por navegador vía VPN

Causa: Puerto 8080 bloqueado.

Solución:

```
sudo ufw allow 8080
sudo ufw allow in on tun0 to any port 8080
```

⚙️ Versión de Elixir 1.14.0

Compatible con las dependencias actuales.

Solo asegúrate de usar versiones recientes de **Plug** y **Cowboy** (≥ 2.6).

✅ 12 Resultado final

La aplicación muestra de forma dinámica:

- Fecha y hora del servidor
 - Versión de Elixir
 - IP del cliente
 - Navegador
 - Resolución de pantalla
 - Sistema operativo
 - Tipo de dispositivo
 - Hora local del cliente (en tiempo real)
-

Resultado visual

El diseño final es limpio, moderno y profesional, con tarjetas centradas, colores suaves y sombras ligeras.

La interfaz es **responsive** y accesible desde cualquier dispositivo.

Ideal para presentarlo como práctica final o exposición técnica.

